



Université
Abdelmalek Essaâdi
École Nationale des Sciences
Appliquées d'Al Hoceima



Rapport

Projet *Job Intelligent*

Plateforme de Centralisation et de Recommandation
d'Offres d'Emploi Data par Intelligence Artificielle

Réalisé par : Khalil Hamimid

Kaoutar El Wahid

Encadrée par : Mme Fadwa Bouhafer

Niveau : ID2

Résumé

Job Intelligent est une plateforme complète de recherche d'emploi dans le domaine de la data, conçue pour répondre à un besoin concret : la dispersion des offres d'emploi spécialisées sur de multiples plateformes hétérogènes. Le projet repose sur une architecture de données moderne (pipeline Medallion Bronze–Silver–Gold), une orchestration robuste via Apache Airflow, et un moteur de recommandation basé sur le traitement du langage naturel (NLP) utilisant TF-IDF et la similarité cosinus.

Le système collecte automatiquement les offres d'emploi depuis **LinkedIn** (via scraping Selenium) et **France Travail** (via API officielle OAuth2), les nettoie et les normalise à travers un pipeline ETL multi-couches, puis les expose via une API REST FastAPI et une interface React intuitive. Un tableau de bord Power BI complète l'écosystème en offrant des visualisations analytiques des tendances du marché.

Ce rapport présente en détail l'ensemble de l'architecture technique, les choix de conception, les implémentations réalisées, ainsi que les résultats obtenus.

Table des matières

1	Introduction et Contexte du Projet	5
1.1	Le Marché de la Data en Pleine Croissance	5
1.2	La Problématique Identifiée	5
1.3	Les Objectifs du Projet <i>Job Intelligent</i>	6
1.3.1	Centralisation des Sources	6
1.3.2	Recommandation Intelligente	6
1.3.3	Précision du Matching	6
1.3.4	Robustesse et Scalabilité	6
2	Architecture Globale et Flux de Données	6
2.1	Vue d'Ensemble de l'Écosystème	7
2.2	Les Sources d'Extraction de Données	8
2.2.1	LinkedIn — Scraping via Selenium	8
2.2.2	France Travail — API Officielle OAuth2	9
2.3	Orchestration avec Apache Airflow	10
2.3.1	Rôle de l'Orchestrateur	10
2.3.2	Le Graphe de Dépendances du DAG	10
2.4	La Couche de Consommation des Données	11
2.4.1	API REST FastAPI	11
2.4.2	Interface React (Frontend)	12
2.4.3	Export pour Power BI	13
3	Data Engineering : L'Architecture Medallion	13
3.1	Le Concept de l'Architecture Medallion	13
3.2	Couche Bronze — Les Données Brutes	14
3.2.1	Philosophie de la Couche Bronze	14
3.2.2	Tables Bronze dans PostgreSQL	14
3.3	Couche Silver — Les Données Nettoyées	15
3.3.1	Philosophie de la Couche Silver	15
3.3.2	Les Transformations Appliquées	15
3.4	Couche Gold — Les Données Métier	17
3.4.1	Philosophie de la Couche Gold	17

3.4.2	Déduplication Globale	17
4	Modélisation Multidimensionnelle — Le Schéma en Étoile	18
4.1	Du Modèle Plat au Modèle Décisionnel	18
4.2	Le Schéma en Étoile de Job Intelligent	19
4.2.1	La Table de Faits — fact_jobs	19
4.2.2	Les Tables de Dimensions	19
4.3	Avantages du Schéma en Étoile pour la BI	21
5	Intelligence Artificielle : Le Moteur NLP	21
5.1	Les Limites de la Recherche Classique par Mots-Clés	22
5.2	Préparation et Extraction du Texte du CV	22
5.3	La Vectorisation TF-IDF	23
5.3.1	Principe du TF-IDF	23
5.3.2	Application dans Job Intelligent	23
5.4	Le Calcul de Similarité Cosinus	24
5.4.1	Principe Géométrique	24
5.4.2	Calcul du Score de Match	25
5.5	Évaluation des Performances du Moteur NLP	26
6	Dashboarding et Business Intelligence	26
6.1	Architecture de la Couche de Visualisation	26
6.2	KPIs et Métriques Clés du Dashboard	27
6.2.1	Métriques de Volume	27
6.2.2	Métriques de Distribution	27
6.2.3	Métriques de Compétences	27
6.3	Insights Clés et Résultats Observés	28
7	Infrastructure Technique et DevOps	29
7.1	Containerisation avec Docker Compose	29
7.1.1	Démarrage de l'Infrastructure	29
7.1.2	Monitoring des Services	30
7.2	Gestion de la Sécurité	31
7.2.1	Variables d'Environnement et Secrets	31
7.2.2	Génération de la Clé Fernet	31
7.2.3	Authentification Utilisateur	32

7.3	Le Développement Frontend	32
8	Démonstration Fonctionnelle	33
8.1	Scénario 1 — Exécution du Pipeline ETL	33
8.2	Scénario 2 — Recommandation Personnalisée par CV	34
8.3	Scénario 3 — Consultation du Dashboard Power BI	35
9	Conclusion	35
9.1	Bilan du Projet	35
	Annexe — Technologies Utilisées	38

I. Introduction et Contexte du Projet

1.1 Le Marché de la Data en Pleine Croissance

Le secteur de la data connaît depuis plusieurs années une expansion remarquable. Les entreprises, qu'elles soient issues du secteur technologique, financier, de la santé ou de l'industrie, cherchent massivement à recruter des profils spécialisés : Data Engineers, Data Scientists, Data Analysts, MLOps Engineers, et bien d'autres encore.

Cette croissance engendre mécaniquement une multiplication des offres d'emploi sur des plateformes diverses. **LinkedIn**, **France Travail** (anciennement Pôle Emploi), **Indeed**, **Welcome to the Jungle** et bien d'autres agrégateurs publient quotidiennement des centaines d'offres, chacune avec son propre format, sa propre nomenclature de compétences et ses propres critères de présentation.

1.2 La Problématique Identifiée

L'hétérogénéité des formats et la dispersion des offres complexifient considérablement la recherche d'emploi pour les candidats. Trois problèmes majeurs ont été identifiés :

- **La dispersion des sources** : Les offres d'emploi sont réparties sur de multiples plateformes sans point d'accès centralisé. Un candidat souhaitant une vision exhaustive du marché doit consulter manuellement chaque plateforme, ce qui est chronophage et inefficace.
- **Le manque de standardisation** : Les intitulés de postes et les compétences exigées varient énormément d'une offre à l'autre. Un même poste peut être intitulé « Data Engineer », « Ingénieur Données », « Data Pipeline Developer » ou encore « ETL Developer » selon les entreprises, rendant les recherches par mots-clés peu fiables.
- **La recherche non personnalisée** : Les plateformes existantes proposent des recherches par mots-clés simples, sans tenir compte du profil réel du candidat, de ses compétences, de son expérience ou de ses aspirations. Le

résultat est une liste d'offres souvent peu pertinente, que le candidat doit trier manuellement.

1.3 Les Objectifs du Projet *Job Intelligent*

Face à cette problématique, le projet *Job Intelligent* a été conçu avec quatre objectifs principaux :

1.3.1 Centralisation des Sources

Rassembler les offres issues de différentes plateformes en un **point d'accès unique**. L'utilisateur ne devrait avoir qu'une seule interface à consulter pour avoir une vision complète du marché de l'emploi data, au lieu de devoir jongler entre plusieurs sites.

1.3.2 Recommandation Intelligente

Développer un système basé sur l'**analyse textuelle** pour personnaliser l'expérience de recherche. Concrètement, cela signifie permettre au candidat de soumettre son CV et d'obtenir en retour une liste d'offres classées par pertinence selon la proximité TF-IDF/cosinus entre son profil et les offres disponibles.

1.3.3 Précision du Matching

Améliorer l'adéquation entre les compétences du candidat et les exigences de l'offre. Le moteur de recommandation compare les termes présents dans le titre saisi, le CV, les titres d'offres, les compétences extraites et les catégories métier afin de calculer un score de compatibilité.

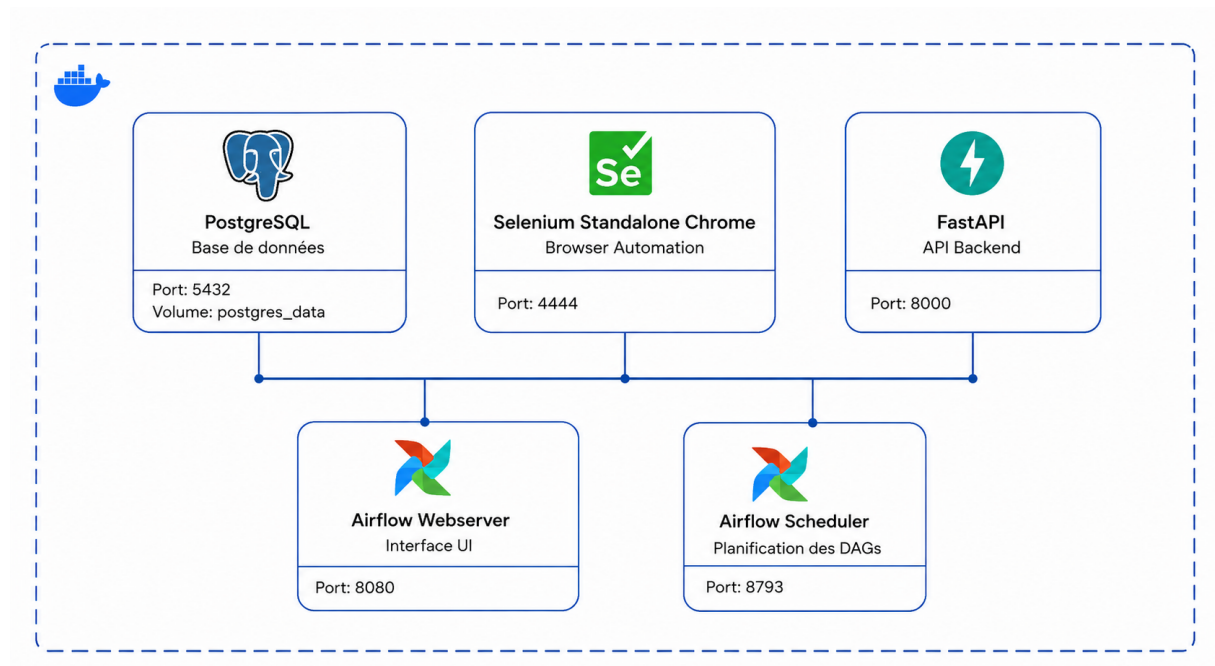
1.3.4 Robustesse et Scalabilité

Assurer la **performance et l'évolutivité** de l'application grâce à une architecture scalable. Le système doit être capable d'absorber l'ajout de nouvelles sources de données et l'augmentation du volume d'offres sans dégradation des performances.

II. Architecture Globale et Flux de Données

2.1 Vue d'Ensemble de l'Écosystème

L'architecture de *Job Intelligent* repose sur une approche **microservices conteneurisée** avec Docker Compose. Chaque composant fonctionnel est isolé dans son propre conteneur, garantissant l'indépendance des services, la facilité de déploiement et la reproductibilité de l'environnement.



L'écosystème se compose des services principaux suivants :

Service	Conteneur	Rôle
PostgreSQL	job_postgres	Base de données principale hébergeant toutes les couches (Bronze, Silver, Gold)
Selenium Chrome	job_selenium	Navigateur headless pour le scraping LinkedIn
FastAPI	job_api	API REST exposant les données et le moteur NLP
Airflow Webserver	job_airflow_webserver	Interface web de supervision des DAGs
Airflow Scheduler	job_airflow_scheduler	Ordonnanceur des tâches ETL

Service	Conteneur	Rôle
Frontend (Vite/- React)	-	Interface utilisateur (développement local)

Table 1 – *Récapitulatif des services de l'architecture Job Intelligent*

2.2 Les Sources d'Extraction de Données

Le premier maillon de la chaîne est la collecte des données brutes depuis les plateformes d'emploi. Deux sources ont été intégrées, chacune avec sa propre méthode d'accès.

2.2.1 LinkedIn — Scraping via Selenium

LinkedIn ne propose pas d'API publique gratuite pour l'accès aux offres d'emploi. La stratégie retenue est donc le **web scraping** via **Selenium WebDriver** en mode « headless » (sans interface graphique visible).

Le conteneur `job_selenium` embarque un navigateur Google Chrome piloté par Selenium Grid. Le scraper `scrapers/scrapping_linkedin.py` pilote ce navigateur pour :

- Naviguer vers les pages de recherche d'offres LinkedIn ciblées sur les métiers de la data.
- Faire défiler la page pour charger dynamiquement les offres (LinkedIn utilise un chargement infini JavaScript).
- Extraire les données structurées de chaque offre : titre, entreprise, localisation, date de publication, URL, mot-clé de recherche et métadonnées disponibles.
- Écrire les données brutes dans la table `bronze_linkedin` de PostgreSQL via la tâche Airflow dédiée.

Technique : L'insertion en base est réalisée de manière **idempotente** grâce à la clause SQL `ON CONFLICT (job_url) DO NOTHING`. Cela signifie que si une offre a déjà été collectée lors d'une exécution précédente, elle n'est pas dupliquée. Cette approche garantit l'intégrité des données même en cas d'exécutions répétées du pipeline.

Listing 1 – *Stratégie d'insertion idempotente pour la couche Bronze*

```
INSERT INTO bronze_linkedin
    (title, company, location, date_posted, job_url,
     search_keyword, scraped_at, salary, contract_type,
     source)
VALUES
    (:title, :company, :location, :date_posted, :
     job_url,
     :search_keyword, :scraped_at, :salary, :
     contract_type, :source)
ON CONFLICT (job_url) DO NOTHING;
```

2.2.2 France Travail — API Officielle OAuth2

Contrairement à LinkedIn, France Travail met à disposition une **API officielle** documentée et accessible via authentification **OAuth2**. Cette approche est nettement plus fiable et moins fragile que le scraping, car elle est officiellement supportée par la plateforme.

Le scraper `scrapers/scrapping_france_travail.py` effectue les étapes suivantes :

1. **Authentification :** Envoi des identifiants `FT_CLIENT_ID` et `FT_CLIENT_SECRET` (stockés dans le fichier `.env`) à l'endpoint OAuth2 de France Travail pour obtenir un *access token* temporaire.
2. **Requête :** Utilisation du token pour interroger l'API de recherche d'offres avec des filtres sur les métiers de la data.
3. **Stockage :** Insertion des résultats dans la table `bronze_france_travail`.

```
FT_CLIENT_ID_api=votre_client_id
FT_CLIENT_SECRET_api=votre_client_secret
```

Listing 2 – Configuration des credentials France Travail dans `.env`

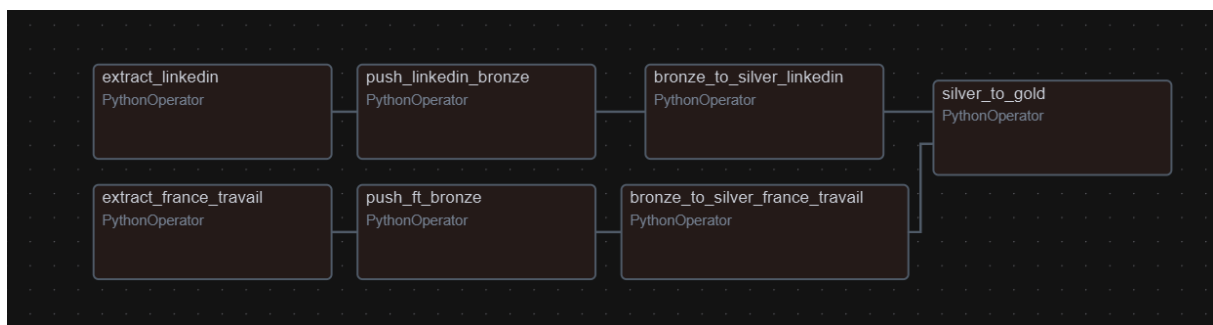
Important : Les variables d'environnement sont nommées `FT_CLIENT_ID_api` et `FT_CLIENT_SECRET_api` dans le fichier `.env`, mais Docker Compose les remappe automatiquement en `FT_CLIENT_ID` et `FT_CLIENT_SECRET` à l'intérieur des conteneurs. Cette distinction est essentielle pour éviter les erreurs d'authentification.

2.3 Orchestration avec Apache Airflow

2.3.1 Rôle de l'Orchestrateur

Apache Airflow est le **chef d'orchestre** de tout le pipeline de données. Il planifie, exécute et supervise l'ensemble des tâches ETL de manière automatique, en respectant les dépendances entre les tâches et en gérant les erreurs.

Le DAG (Directed Acyclic Graph) principal est défini dans `dags/job_scraping_dag.py` avec l'identifiant `daily_job_scraping`. Il est planifié pour s'exécuter **quotidiennement à 12h00 UTC**.



2.3.2 Le Graphe de Dépendances du DAG

Le pipeline suit un **flux parallèle** pour les deux sources de données, qui convergent ensuite vers une étape de fusion :

Listing 3 – Structure du DAG `job_scraping_dag.py`

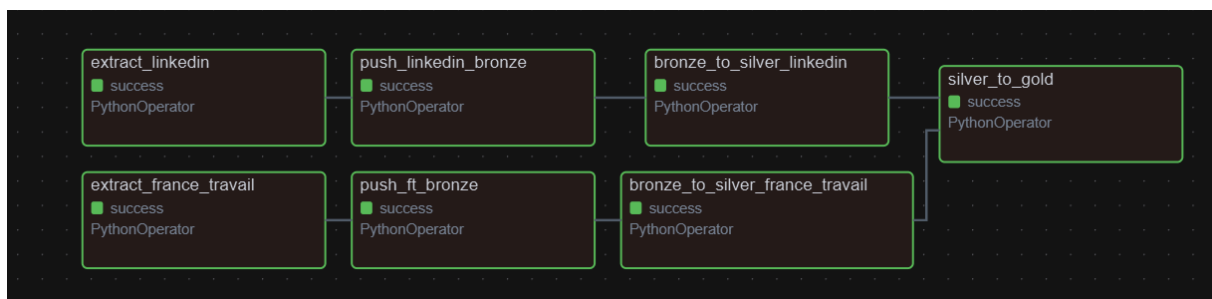
```
# Flux LinkedIn
extract_linkedin >> push_linkedin_bronze >>
    bronze_to_silver_linkedin
```

```
# Flux France Travail
extract_france_travail >> push_ft_bronze >>
    bronze_to_silver_france_travail

# Convergence vers Gold
[bronze_to_silver_linkedin,
    bronze_to_silver_france_travail] >> silver_to_gold
```

Ce graphe garantit que :

- Les extractions LinkedIn et France Travail s'exécutent en **parallèle**, réduisant le temps total d'exécution.
- La transformation vers la couche Gold n'est déclenchée que lorsque **les deux** branches Silver sont terminées avec succès.
- En cas d'échec de l'une des branches, Airflow le signale clairement et permet une reprise ciblée.



2.4 La Couche de Consommation des Données

Une fois les données traitées et disponibles dans la couche Gold, elles sont exposées à travers plusieurs canaux :

2.4.1 API REST FastAPI

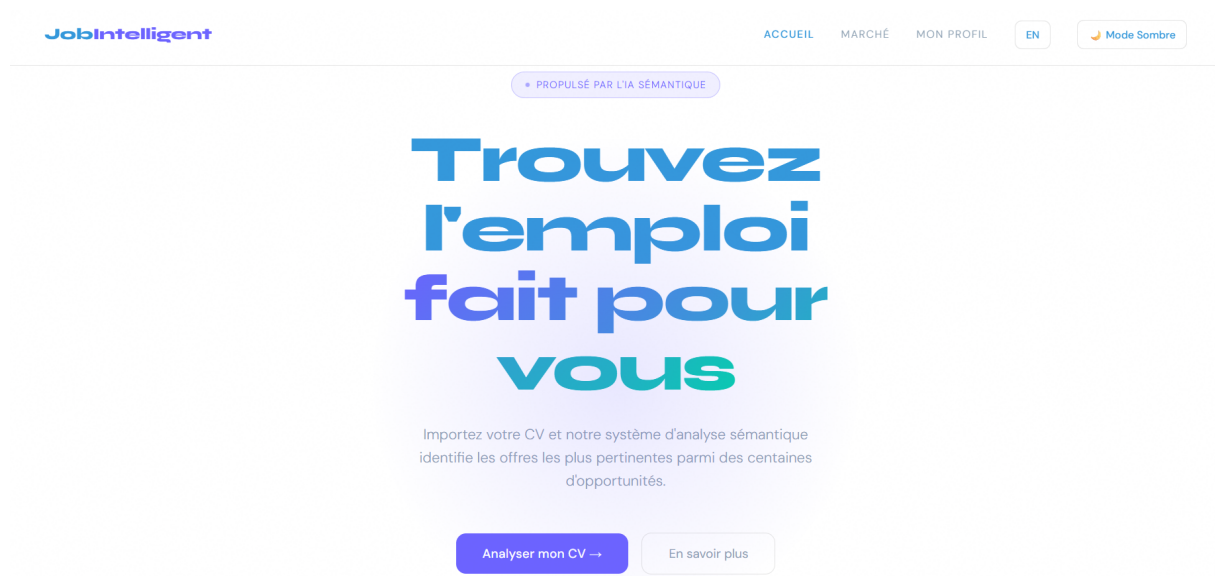
Le service `api/api.py` expose les données via une **API REST** construite avec FastAPI. Les principaux endpoints sont :

Endpoint	Méthode	Description
/health	GET	Vérification de l'état de l'API et de la connexion PostgreSQL
/jobs	GET	Liste paginée de toutes les offres Gold, avec filtres optionnels
/jobs/search/{keyword}	GET	Recherche full-text par mot-clé dans les offres
/stats	GET	Statistiques agrégées sur le marché
/stats/skills	GET	Classement des compétences les plus demandées
/recommend	POST	Recommandation personnalisée par titre, ville et CV PDF
/register	POST	Création d'un compte utilisateur
/login	POST	Authentification utilisateur
/analysis/{user_id}	GET	Récupération de la dernière analyse sauvegardée

Table 2 – Endpoints principaux de l'API FastAPI Job Intelligent

2.4.2 Interface React (Frontend)

Le frontend est une **Single Page Application (SPA)** développée avec **Vite + React**. Il offre une expérience utilisateur intuitive avec gestion de session (persistée dans `localStorage`) et la possibilité d'uploader un CV au format PDF pour obtenir des recommandations.



2.4.3 Export pour Power BI

À la fin de chaque exécution du pipeline, la tâche `silver_to_gold` génère automatiquement deux fichiers d'export dans le répertoire `scrapers/data/` :

- `gold_jobs.csv` — Format tabulaire pour intégration dans Power BI Desktop.
- `gold_jobs.json` — Format structuré pour des intégrations API ou NLP.

III. Data Engineering : L'Architecture Medallion

3.1 Le Concept de l'Architecture Medallion

L'architecture Medallion est un **modèle de conception de données** structuré en trois couches successives, chacune portant le nom d'un métal précieux. Ce modèle, popularisé par **Databricks**, est aujourd'hui un standard de l'industrie pour la construction de pipelines de données robustes.

Définition : L'**Architecture Medallion** (ou Lakehouse Architecture) est un paradigme de modélisation des données qui organise le flux de transformation en trois couches distinctes : Bronze (données brutes), Silver (données nettoyées) et Gold (données métier agrégées). Chaque couche représente un niveau croissant de qualité et de valeur ajoutée.

L'objectif principal de cette architecture est de garantir la **qualité**, la **fiabilité** et l'**évolutivité** des données tout au long de leur cycle de vie. Chaque couche ayant une fonction spécifique, le débogage est simplifié : si un problème survient dans les données finales, il est facile de remonter à la couche défaillante.

3.2 Couche Bronze — Les Données Brutes

3.2.1 Philosophie de la Couche Bronze

La couche Bronze est le **point d'entrée** de toutes les données dans le système. Sa philosophie est simple : *collecter tout, modifier rien*. Les données sont stockées telles qu'elles arrivent des sources, sans aucune transformation ni nettoyage.

Cette approche présente plusieurs avantages :

- **Traçabilité complète** : Il est toujours possible de retrouver exactement ce que la source a fourni, même des mois après la collecte.
- **Reprise sur erreur** : En cas de bug dans les étapes de transformation, il suffit de relire la couche Bronze pour recalculer les couches supérieures, sans avoir à relancer le scraping.
- **Flexibilité** : De nouveaux champs peuvent être ajoutés aux transformations ultérieures sans nécessiter une nouvelle collecte des données.

3.2.2 Tables Bronze dans PostgreSQL

Deux tables Bronze sont maintenues dans PostgreSQL :

Listing 4 – *Structure simplifiée des tables Bronze*

```
-- Table Bronze LinkedIn
CREATE TABLE bronze_linkedin (
    id          SERIAL PRIMARY KEY,
    title       TEXT,
    company     TEXT,
    location    TEXT,
    date_posted TEXT,
    job_url     TEXT UNIQUE,
    search_keyword TEXT,
```

```
        scraped_at      TEXT,
        salary          TEXT,
        contract_type   TEXT,
        source          TEXT DEFAULT 'linkedin',
        ingested_at     TIMESTAMP DEFAULT NOW()
    );

-- Table Bronze France Travail
CREATE TABLE bronze_france_travail (
    id                  SERIAL PRIMARY KEY,
    title              TEXT,
    company            TEXT,
    location           TEXT,
    date_posted        TEXT,
    job_url            TEXT UNIQUE,
    search_keyword     TEXT,
    scraped_at         TEXT,
    salary             TEXT,
    contract_type      TEXT,
    source             TEXT DEFAULT 'france_travail',
    ingested_at        TIMESTAMP DEFAULT NOW()
);
```

3.3 Couche Silver — Les Données Nettoyées

3.3.1 Philosophie de la Couche Silver

La couche Silver est la **couche de nettoyage et de normalisation**. Elle prend les données brutes de la couche Bronze et les transforme en données fiables, cohérentes et exploitables. C'est ici que la majorité du travail de *data quality* est réalisé.

3.3.2 Les Transformations Appliquées

Le script `scrapers/eda_pipeline.py` applique les transformations suivantes lors du passage Bronze → Silver :

- **Nettoyage des champs textuels** : Conversion des champs principaux en chaînes propres avec suppression des valeurs nulles et des espaces superflus.
- **Standardisation des intitulés de postes** : Application d'un dictionnaire de correspondances par expressions régulières pour regrouper les variantes d'un même poste. Par exemple, « Data Engineer » ou « Ingénieur Data » sont mappés vers une catégorie normalisée.
- **Normalisation des localisations** : Les localisations contenant plusieurs niveaux géographiques sont simplifiées en conservant principalement la première partie, généralement la ville.
- **Normalisation des dates** : Les dates de publication, fournies dans des formats variés selon les sources, sont converties en type **DATE** lorsque le parsing est possible.
- **Extraction des compétences** : Un module d'extraction recherche une liste prédéfinie de compétences (Python, SQL, Spark, Docker, AWS, etc.) dans le titre brut et le nom de l'entreprise afin de peupler le champ **skills**.
- **Catégorisation des offres** : Chaque offre est assignée à une catégorie métier large (Data & AI, DevOps & Cloud, Cybersecurity, Software Dev, Management, IT & Systems ou Other) sur la base de l'analyse de son titre standardisé.

Listing 5 – Exemple de normalisation des intitulés de postes dans `eda_pipeline.py`

```
def standardize_title(title: str) -> str:
    t = title.lower().strip()
    for pattern, standard in TITLE_MAP.items():
        if re.match(pattern, t):
            return standard
    return title.title()
```

Stratégie de remplacement : Les tables Silver sont **entièrement remplacées** à chaque exécution du pipeline (*full refresh*). Cette stratégie, bien que moins optimale en termes de ressources que les insertions incrémentales, garantit la cohérence totale des données et simplifie la logique de transformation.

3.4 Couche Gold — Les Données Métier

3.4.1 Philosophie de la Couche Gold

La couche Gold est la **couche de présentation**. Elle contient les données sous leur forme la plus raffinée, prêtes à être consommées par les applications finales : API, dashboards BI, moteur NLP. C'est la version « production » des données.

3.4.2 Déduplication Globale

La tâche `silver_to_gold` fusionne les deux tables Silver (LinkedIn et France Travail) et effectue une **déduplication globale** par `job_url`. En effet, certaines offres peuvent apparaître sur les deux plateformes ; la déduplication garantit qu'elles n'apparaissent qu'une seule fois dans la couche Gold.

Listing 6 – Logique de fusion et déduplication Silver → Gold

```
import pandas as pd

def silver_to_gold():
    # Lecture des deux sources Silver
    df_linkedin = read_table('silver_linkedin')
    df_ft = read_table('silver_france_travail')

    # Fusion et déduplication par URL
    df_gold = pd.concat([df_linkedin, df_ft],
                        ignore_index=True)
    df_gold = df_gold.drop_duplicates(subset=['job_url'],
                                     keep='first')

    # Écriture dans la table Gold (remplacement complet)
```

```

    )

    df_gold.to_sql('gold_jobs', engine, if_exists='
        replace', index=False)

    # Export des fichiers
    df_gold.to_csv('scrapers/data/gold_jobs.csv', index
        =False)
    df_gold.to_json('scrapers/data/gold_jobs.json',
        orient='records', force_ascii=False)

    print(f"Gold layer: {len(df_gold)} offres uniques
        chargées.")

```

```

[2026-05-17, 09:24:45 UTC] {logging_mixin.py:190} INFO -
By category:
category
Data & AI          608
Other              190
Software Dev       129
Management         33
DevOps & Cloud     20
IT & Systems       12
Cybersecurity       7
[2026-05-17, 09:24:45 UTC] {logging_mixin.py:190} INFO -
Top locations:
location
Casablanca         252
Rabat              75
Casablanca Metropolitan Area  56
Morocco            51
Mecouar            50
[2026-05-17, 09:24:45 UTC] {logging_mixin.py:190} INFO -
[2026-05-17, 09:24:45 UTC] {logging_mixin.py:190} INFO -
--- Medallion DB Stats ---
[2026-05-17, 09:24:45 UTC] {logging_mixin.py:190} INFO - bronze_linkedin      : 788 rows
[2026-05-17, 09:24:45 UTC] {logging_mixin.py:190} INFO - bronze_france_travail : 461 rows
[2026-05-17, 09:24:45 UTC] {logging_mixin.py:190} INFO - silver_linkedin       : 630 rows
[2026-05-17, 09:24:45 UTC] {logging_mixin.py:190} INFO - silver_france_travail : 369 rows
[2026-05-17, 09:24:45 UTC] {logging_mixin.py:190} INFO - gold_jobs             : 999 rows
[2026-05-17, 09:24:45 UTC] {logging_mixin.py:190} INFO -
[2026-05-17, 09:24:45 UTC] {python.py:240} INFO - Done. Returned value was: None
[2026-05-17, 09:24:45 UTC] {taskinstance.py:340} ▶ Post task execution logs

```

IV. Modélisation Multidimensionnelle — Le Schéma en Étoile

4.1 Du Modèle Plat au Modèle Décisionnel

La couche Gold contient initialement les données sous forme d'une **table plate** (*flat table*) : une ligne par offre d'emploi, avec tous les attributs dans une seule

table. Si ce format est parfait pour le NLP et l'API, il n'est pas optimal pour l'analyse décisionnelle et le reporting.

Pour répondre aux besoins du dashboard Power BI, une transformation supplémentaire organise les données selon un **Schéma en Étoile** (*Star Schema*), le modèle standard de l'informatique décisionnelle (BI).

4.2 Le Schéma en Étoile de Job Intelligent

4.2.1 La Table de Faits — fact_jobs

La table de faits **fact_jobs** est au centre du schéma. Elle représente les offres d'emploi analysées et contient les mesures principales ainsi que les clés permettant de relier chaque offre aux différentes dimensions du modèle décisionnel.

Listing 7 – *Structure simplifiée de la table de faits fact_jobs*

```
CREATE TABLE fact_jobs (  
    job_id          INT PRIMARY KEY,  
    month_id        INT REFERENCES dim_month(month_id),  
    company_id      INT REFERENCES dim_company(company_id),  
    title_id        INT REFERENCES dim_title(title_id),  
    daily_job_id    INT REFERENCES dim_daily_job(  
        daily_job_id),  
    location_id     INT REFERENCES dim_location(  
        location_id),  
    match_score     FLOAT,  
    source          TEXT  
);
```

4.2.2 Les Tables de Dimensions

Cinq tables de dimensions gravitent autour de la table de faits, conformément au schéma étoile présenté dans le dashboard :

dim_month — La dimension mensuelle permet d'analyser l'évolution des offres par mois, d'identifier les périodes de forte activité et de suivre les tendances temporelles globales.

Listing 8 – *Structure simplifiée de dim_month*

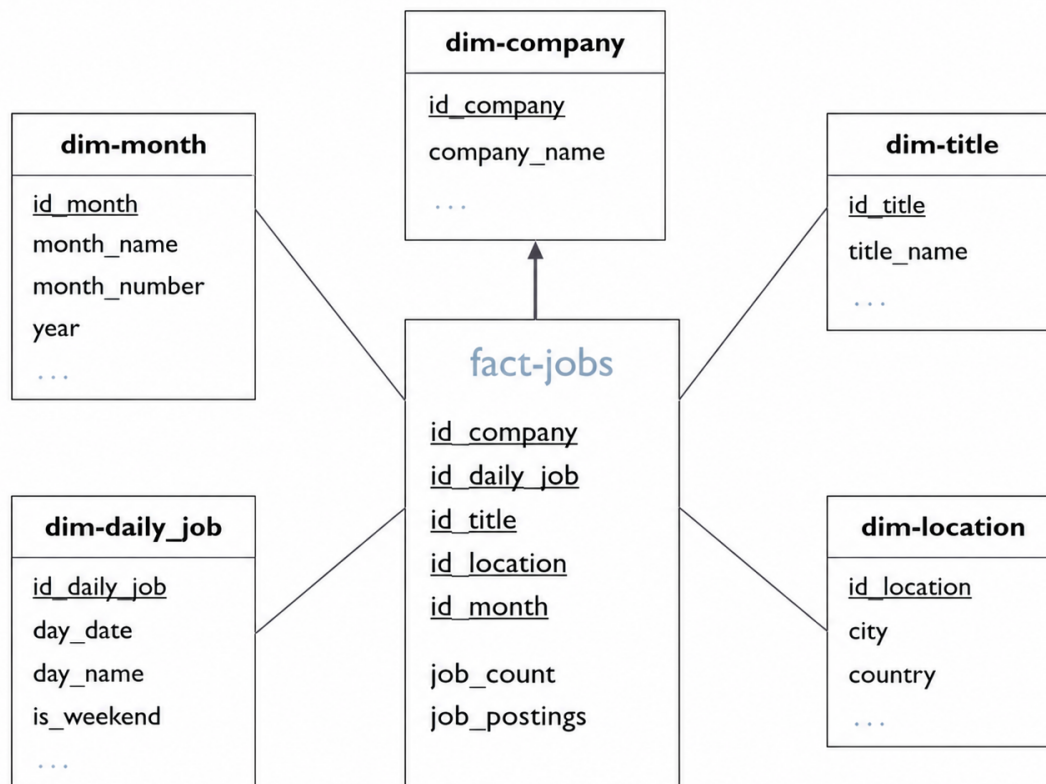
```
CREATE TABLE dim_month (  
    month_id    INT PRIMARY KEY,  
    month       INT,  
    month_name  TEXT,  
    year        INT  
);
```

dim_company — La dimension entreprise permet d’identifier les recruteurs les plus actifs et de comparer le volume d’offres publié par chaque société.

dim_title — La dimension titre permet d’analyser la demande par intitulé ou famille de poste, par exemple Data Engineer, Data Scientist, Data Analyst ou MLOps Engineer.

dim_daily_job — La dimension journalière permet de suivre les offres au niveau quotidien, notamment pour analyser les nouvelles publications et l’activité récente du marché.

dim_location — La dimension géographique permet d’analyser la répartition des offres par ville, région ou pays.



4.3 Avantages du Schéma en Étoile pour la BI

Le schéma en étoile présente plusieurs avantages décisifs pour l'analyse décisionnelle :

- **Performance des requêtes** : Les jointures entre la table de faits et les dimensions sont simples et directes, ce qui permet à des outils comme Power BI de générer des requêtes SQL optimisées.
- **Intuitivité** : Le modèle est compréhensible par des utilisateurs non techniques, ce qui facilite la création de rapports en self-service.
- **Flexibilité analytique** : Il est possible de slicer et dicer les données selon n'importe quelle combinaison de dimensions (offres par mois ET par ville ET par catégorie, par exemple).

V. Intelligence Artificielle : Le Moteur NLP

5.1 Les Limites de la Recherche Classique par Mots-Clés

Avant de comprendre la solution apportée par le moteur NLP, il est important de comprendre pourquoi la recherche classique par mots-clés est insuffisante dans ce contexte.

Une recherche par mots-clés fonctionne par correspondance exacte (ou approchée) : si l'utilisateur tape « Python », seules les offres contenant le mot « Python » dans leur texte sont retournées. Cette approche ignore le **sens sémantique** et présente de nombreuses limites :

- **Synonymes manqués** : Une offre mentionnant « Scikit-learn » ou « sklearn » mais pas « Machine Learning » ne serait pas retournée pour une recherche « Machine Learning ».
- **Contexte ignoré** : Le terme « Python » peut désigner le langage de programmation ou le serpent. Le moteur de recherche classique ne fait pas la distinction.
- **Profil global non pris en compte** : L'utilisateur ne peut pas soumettre son CV complet pour une recherche globale ; il doit formuler des mots-clés, ce qui est un exercice de reformulation de son propre profil.

5.2 Préparation et Extraction du Texte du CV

La première étape du moteur de recommandation est l'**extraction du texte** depuis le CV de l'utilisateur. Le CV est soumis au format PDF via l'endpoint `/recommend` de l'API.

Le module `PyPDF2` est utilisé pour extraire le contenu textuel du PDF. Ce texte est ensuite concaténé avec les autres informations du profil de l'utilisateur pour former un **document de profil unifié**.

Listing 9 – *Extraction de texte depuis un CV PDF avec PyPDF2*

```
import PyPDF2
import io
```

```
def extract_text_from_pdf(pdf_bytes: bytes) -> str:
    """Extrait le texte brut d'un fichier PDF."""
    pdf_reader = PyPDF2.PdfReader(io.BytesIO(pdf_bytes))
    full_text = ""
    for page in pdf_reader.pages:
        full_text += page.extract_text() + "\n"
    return full_text.strip()
```

5.3 La Vectorisation TF-IDF

5.3.1 Principe du TF-IDF

Le **TF-IDF** (Term Frequency–Inverse Document Frequency) est une méthode statistique qui transforme un texte en un **vecteur numérique** représentant l'importance de chaque mot dans ce texte, relativement à un corpus de référence.

La formule du TF-IDF est :

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

Où :

- $\text{TF}(t, d)$ = Fréquence du terme t dans le document d (combien de fois le mot apparaît dans ce texte)
- $\text{IDF}(t, D) = \log\left(\frac{|D|}{|\{d \in D: t \in d\}|}\right)$ = Inverse de la fréquence documentaire (pénalise les mots très courants comme « le », « de », « et »)

5.3.2 Application dans Job Intelligent

Dans notre système, le corpus de référence D est l'ensemble des offres d'emploi retournées par la requête sur la couche Gold, éventuellement filtrées par titre ou localisation. Chaque offre est représentée par la combinaison du titre, des compétences et de la catégorie. Le texte du CV et du profil utilisateur est ajouté au même corpus avant l'entraînement du vectoriseur.

Listing 10 – Vectorisation TF-IDF dans *api/api.py*

```

from sklearn.feature_extraction.text import
    TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Création du texte NLP pour chaque offre
nlp_text = (
    df_jobs['title'].fillna('') + ' ' +
    df_jobs['skills'].fillna('') + ' ' +
    df_jobs['category'].fillna('')
).str.lower()

# Texte utilisateur : titre saisi + texte extrait du CV
user_text = f"{title} {cv_text}".lower()

# Vectorisation commune offres + utilisateur
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(nlp_text.tolist()
    () + [user_text])

user_vector = tfidf_matrix[-1]
job_vectors = tfidf_matrix[:-1]

```

5.4 Le Calcul de Similarité Cosinus

5.4.1 Principe Géométrique

Une fois le CV et les offres transformés en vecteurs dans le même espace TF-IDF, la **similarité cosinus** est utilisée pour mesurer leur proximité sémantique.

La similarité cosinus mesure le **cosinus de l'angle** entre deux vecteurs. Un angle proche de 0° (cosinus proche de 1) indique une grande similarité ; un angle proche de 90° (cosinus proche de 0) indique l'absence de similarité.

$$\text{cosine_similarity}(\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \times \|\vec{B}\|} = \cos(\theta)$$

L'avantage de cette mesure sur la distance euclidienne classique est qu'elle dépend principalement de l'orientation des vecteurs, et donc de la proximité des termes importants, plutôt que de leur volume brut.

5.4.2 Calcul du Score de Match

Listing 11 – *Calcul du score de match CV-Offre avec similarité cosinus*

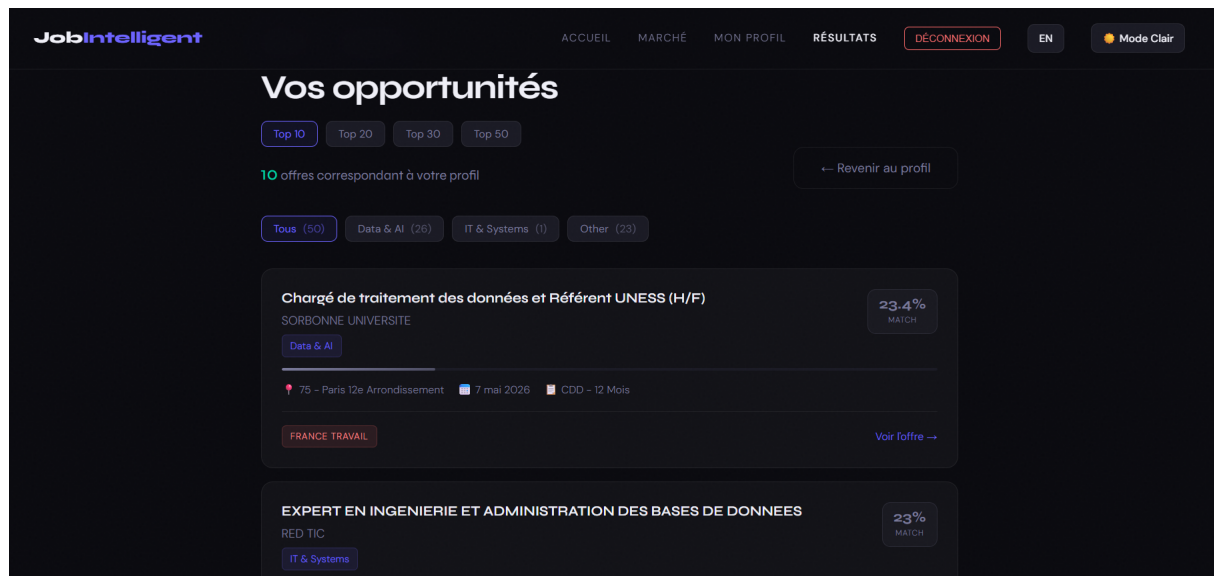
```
# Texte NLP de chaque offre
nlp_text = (
    df_jobs['title'].fillna('') + ' ' +
    df_jobs['skills'].fillna('') + ' ' +
    df_jobs['category'].fillna('')
).str.lower()

user_text = f"{title} {cv_text}".lower()

# Vectorisation commune offres + profil utilisateur
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(nlp_text.tolist()
    + [user_text])

user_vector = tfidf_matrix[-1]
job_vectors = tfidf_matrix[:-1]
similarities = cosine_similarity(user_vector,
    job_vectors)[0]

# Score en pourcentage et tri décroissant
df_jobs['score'] = (similarities * 100).round(1)
matched_jobs = df_jobs[df_jobs['score'] > 0] \
    .sort_values(by='score', ascending=False) \
    .head(limit)
```



5.5 Évaluation des Performances du Moteur NLP

Le moteur TF-IDF + similarité cosinus, bien que relativement simple, offre de bonnes performances pour ce cas d'usage, pour plusieurs raisons :

- **Domaine spécialisé** : Le corpus est centré sur les offres d'emploi tech/-data, ce qui réduit la variabilité du vocabulaire et améliore la pertinence des correspondances.
- **Simplicité du corpus** : Le texte utilisé pour chaque offre combine principalement le titre, les compétences extraites et la catégorie métier, ce qui rend le calcul rapide et interprétable.
- **Légèreté** : Contrairement aux modèles de transformers (BERT, GPT), TF-IDF + cosinus ne nécessite ni GPU ni modèle lourd, ce qui est adapté à une API temps réel.

VI. Dashboarding et Business Intelligence

6.1 Architecture de la Couche de Visualisation

Le flux de données vers Power BI suit le chemin suivant :

Couche Gold (PostgreSQL) → Fichiers CSV/JSON → Power BI Desktop → Dashboards Interactifs

Le choix de passer par des fichiers intermédiaires (plutôt qu’une connexion directe PostgreSQL–Power BI) est pragmatique : les fichiers CSV/JSON sont générés automatiquement à chaque exécution du pipeline et peuvent être versionnés ou archivés pour un suivi historique.

6.2 KPIs et Métriques Clés du Dashboard

Le dashboard Power BI a été conçu pour répondre à des questions analytiques précises sur le marché de l’emploi data :

6.2.1 Métriques de Volume

- **Nombre total d’offres actives** : Comptage des offres dans la couche Gold, mis à jour quotidiennement.
- **Évolution temporelle** : Graphique montrant l’évolution du nombre d’offres selon les dates disponibles dans la couche Gold.
- **Nouvelles offres dans les 7 derniers jours** : Indicateur de l’activité récente du marché.

6.2.2 Métriques de Distribution

- **Répartition par source** : Camembert montrant la part de LinkedIn vs France Travail dans le total des offres.
- **Top des catégories de postes** : Graphique en barres des catégories les plus recherchées (Data Engineer, Data Scientist, etc.).
- **Répartition géographique** : Carte ou graphique montrant les villes avec le plus d’offres.
- **Top des entreprises recruteuses** : Classement des entreprises les plus actives.

6.2.3 Métriques de Compétences

- **Compétences les plus demandées** : Word cloud ou graphique en barres des technologies les plus mentionnées (Python, SQL, Spark, Azure, AWS, etc.).

- **Analyse des compétences** : Classement des technologies extraites à partir des titres et métadonnées disponibles.



Figure 1 – Dashboard Power BI — Vue principale

6.3 Insights Clés et Résultats Observés

L'analyse des données collectées via le pipeline Job Intelligent a permis de dégager plusieurs insights pertinents sur le marché de l'emploi data :

- **Dominance de Python** : Python apparaît dans la grande majorité des offres de data, quelle que soit la spécialité (Data Engineering, Data Science, Data Analysis). Il s'impose comme la compétence transversale incontournable du secteur.
- **SQL toujours indispensable** : Malgré l'émergence de technologies NoSQL et de frameworks de traitement distribué, SQL reste présent dans une proportion très significative des offres, confirmant sa position de compétence fondamentale.
- **Cloud en forte demande** : Les compétences cloud (AWS, Azure, GCP) sont de plus en plus demandées, reflétant la migration massive des infrastructures data vers le cloud.

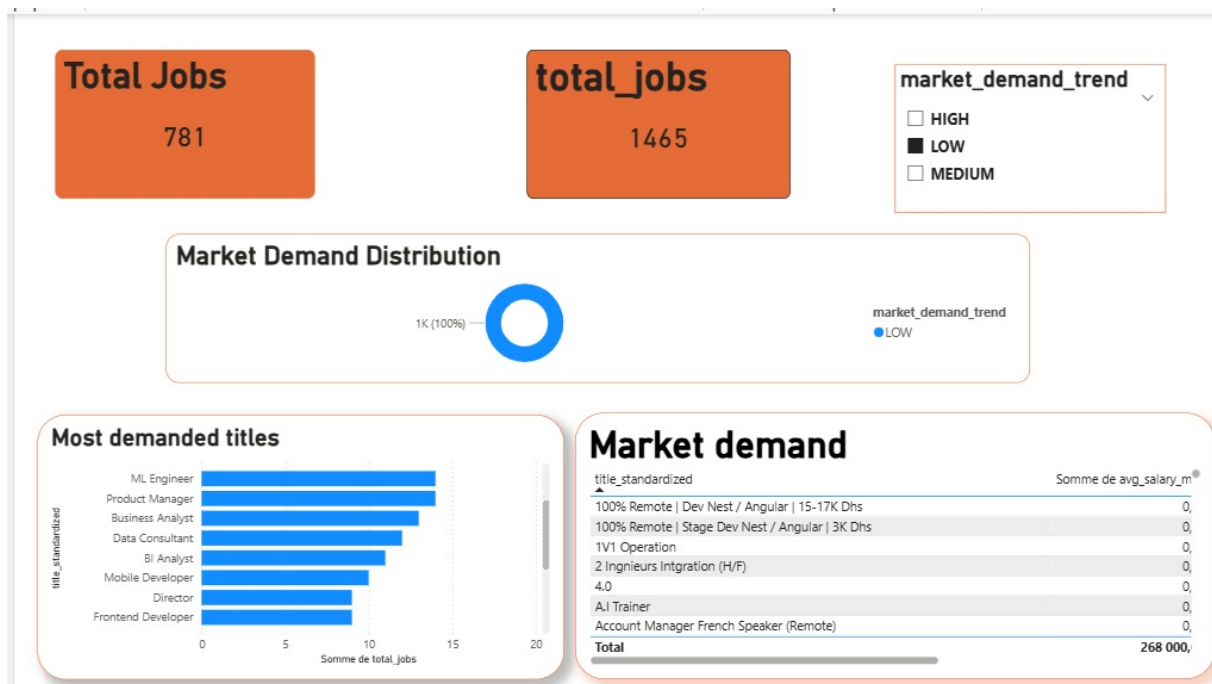


Figure 2 – Dashboard Power BI — Analyse des compétences

- **Analyse géographique** : Les champs de localisation permettent d'identifier les villes les plus représentées dans les données collectées et de comparer les dynamiques par zone.
- **Maroc — marché ciblé** : Pour les offres ciblant le Maroc, les visualisations permettent de suivre la concentration des offres par ville et la demande pour les profils Data Engineer, Data Analyst, Data Scientist ou MLOps.

VII. Infrastructure Technique et DevOps

7.1 Containerisation avec Docker Compose

L'ensemble du projet est conteneurisé avec **Docker** et **Docker Compose**, garantissant la reproductibilité de l'environnement sur n'importe quelle machine.

7.1.1 Démarrage de l'Infrastructure

```
# Initialisation de la base de données Airflow (
    première fois uniquement)
docker compose up airflow-init
# Si le code de sortie est 1 : docker compose down -v
    et réessayer
```

```
# Démarrage complet du stack en arrière-plan
docker compose up -d

# Reconstruction des images après changement de
  dépendances
docker compose build --no-cache

# Arrêt du stack (conservation des données)
docker compose down

# Arrêt complet avec suppression des volumes (reset
  destructif)
docker compose down -v
```

Listing 12 – *Commandes de gestion du stack Docker Compose*

7.1.2 Monitoring des Services

```
# Logs du scheduler Airflow en temps réel
docker compose logs -f airflow-scheduler

# Logs de l'API FastAPI
docker compose logs -f api

# Logs de la base de données PostgreSQL
docker compose logs -f postgres

# Accès direct à la base de données
docker exec -it job_postgres psql -U admin -d
  job_intelligent
```

Listing 13 – *Commandes de surveillance et de débogage*

Nomenclature importante : Il existe une distinction entre les noms de **services Compose** et les noms de **conteneurs Docker**. Par exemple,

le service Compose s'appelle **postgres** mais le conteneur Docker s'appelle **job_postgres**. Utiliser les noms de services pour **docker compose logs** et les noms de conteneurs pour **docker exec**.

7.2 Gestion de la Sécurité

7.2.1 Variables d'Environnement et Secrets

Toutes les informations sensibles (credentials, clés de sécurité) sont stockées dans un fichier **.env** à la racine du projet, qui **ne doit jamais être commité** dans le dépôt Git. Docker Compose charge automatiquement ce fichier.

```
# Credentials France Travail
FT_CLIENT_ID_api=votre_client_id
FT_CLIENT_SECRET_api=votre_client_secret

# Sécurité Airflow
FERNET_KEY=votre_cle_fernet_generee
SECRET_KEY=votre_cle_secrete

# Base de données
POSTGRES_USER=admin
POSTGRES_PASSWORD=votre_mot_de_passe
POSTGRES_DB=job_intelligent
```

Listing 14 – Structure du fichier *.env* (variables requises)

7.2.2 Génération de la Clé Fernet

La clé Fernet est utilisée par Airflow pour chiffrer les connexions et variables sensibles stockées dans sa base de données. Elle doit être générée une seule fois et conservée en lieu sûr :

Listing 15 – Génération d'une clé Fernet (*test.py*)

```
from cryptography.fernet import Fernet
key = Fernet.generate_key()
print(key.decode()) # Copier cette valeur dans .env
                    sous FERNET_KEY
```

7.2.3 Authentification Utilisateur

L'API FastAPI implémente un système d'authentification par mot de passe utilisant le hachage **PBKDF2-HMAC** (Password-Based Key Derivation Function 2). Ce standard de sécurité, recommandé par le NIST, protège les mots de passe des utilisateurs même en cas de compromission de la base de données.

7.3 Le Développement Frontend

Le frontend est une **SPA (Single Page Application)** développée avec la stack moderne **Vite + React**.

```
# Installation des dépendances
cd frontend && npm install

# Serveur de développement avec hot-reload
cd frontend && npm run dev

# Vérification de la qualité du code (ESLint)
cd frontend && npm run lint

# Build de production optimisé
cd frontend && npm run build

# Prévisualisation du build de production
cd frontend && npm run preview
```

Listing 16 – *Workflow de développement frontend*

L'application frontend communique avec l'API FastAPI à l'adresse **http://localhost:8000**. Elle implémente une gestion de session persistée dans **localStorage**, permettant aux utilisateurs de rester connectés entre les sessions.

Job Intelligent ACCUEIL MARCHÉ MON PROFIL RÉSULTATS DÉCONNEXION EN Mode Clair

BONJOUR, KHALIL

Votre profil

Plus votre CV est détaillé, plus les recommandations seront précises.

Une analyse précédente a été trouvée.
Voulez-vous consulter vos résultats sauvegardés ? [Voir les résultats](#)

TITRE DE POSTE RECHERCHÉ (OPTIONNEL) VILLE SOUHAITÉE

Data Engineer, ML Engineer... Meknès, Casablanca...

VOTRE CV (PDF)
Notre IA va extraire vos compétences et expériences directement depuis votre CV.

CV CVdya1.pdf 96 Ko Retirer

[← Retour](#) [Lancer une nouvelle analyse →](#)

VIII. Démonstration Fonctionnelle

8.1 Scénario 1 — Exécution du Pipeline ETL

La démonstration du pipeline ETL complet suit les étapes suivantes :

1. **Démarrage du stack :**

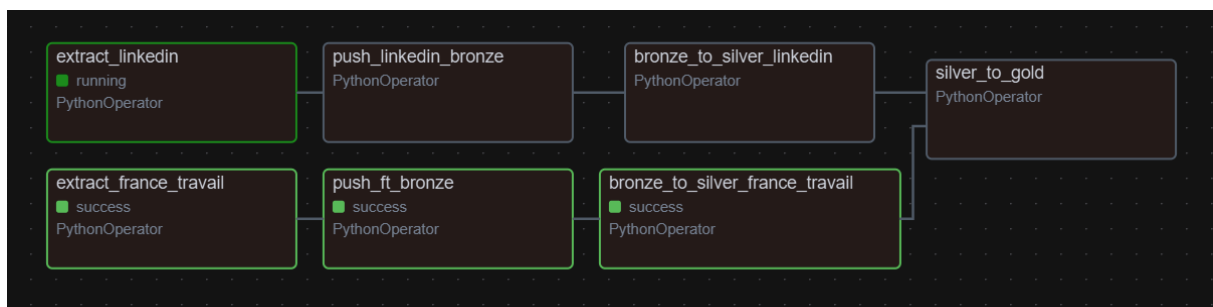
```
docker compose up -d
```

Vérifier que tous les conteneurs sont en état Up avec `docker compose ps`.

2. **Accès à l'interface Airflow :** Ouvrir `http://localhost:8080` et se connecter avec les identifiants `admin / admin`.
3. **Activation du DAG :** Toggler le DAG `daily_job_scraping` en position ON.
4. **Déclenchement manuel :** Cliquer sur le bouton « Trigger DAG » pour lancer immédiatement le pipeline sans attendre l'heure planifiée (12h00 UTC).
5. **Suivi de l'exécution :** Observer en temps réel l'avancement des tâches dans la vue Graph d'Airflow. Les tâches passent successivement de l'état *queued* (orange) à *running* (vert clignotant) puis *success* (vert plein).

6. **Vérification des données** : Se connecter à PostgreSQL pour vérifier les données dans les différentes couches :

```
docker exec -it job_postgres psql -U admin -d
  job_intelligent
-- Vérification des couches
SELECT COUNT(*) FROM bronze_linkedin;
SELECT COUNT(*) FROM silver_linkedin;
SELECT COUNT(*) FROM gold_jobs;
```



layer_table	rows
bronze_linkedin	801
bronze_france_travail	461
silver_linkedin	643
silver_france_travail	369
gold_jobs	1012
(5 rows)	

8.2 Scénario 2 — Recommandation Personnalisée par CV

Ce scénario illustre le flux complet de recommandation d'offres à partir d'un CV :

1. **Création d'un compte** : Via l'interface React, créer un compte utilis-

teur ou se connecter avec un compte existant.

2. **Upload du CV** : Naviguer vers la section « Recommandations » et uploader un CV au format PDF.
3. **Traitement par l'API** : L'API reçoit le PDF, extrait le texte avec PyPDF2, construit le corpus à partir des offres filtrées et du profil utilisateur, puis applique TF-IDF et la similarité cosinus.
4. **Affichage des résultats** : L'interface affiche les offres compatibles, triées par score de match décroissant, avec le score affiché en pourcentage.
5. **Sauvegarde de l'analyse** : Si l'utilisateur est connecté, l'analyse est sauvegardée dans la table `user_analyses` de PostgreSQL, accessible ultérieurement via l'endpoint `/analysis/{user_id}`.

8.3 Scénario 3 — Consultation du Dashboard Power BI

1. Ouvrir Power BI Desktop et charger le fichier `scrapers/data/gold_jobs.csv`.
2. Naviguer entre les différentes pages du rapport : Vue Générale, Analyse Géographique, Analyse des Compétences.
3. Utiliser les filtres interactifs pour explorer les données (filtre par date, par source, par catégorie).

IX. Conclusion

9.1 Bilan du Projet

Le projet *Job Intelligent* répond avec succès à la problématique initiale : simplifier et améliorer la recherche d'emploi dans le domaine de la data grâce à la centralisation automatisée des offres et à la recommandation personnalisée par analyse sémantique.

Sur le plan **technique**, le projet démontre la mise en œuvre cohérente d'un



Figure 3 – Dashboard Power BI — Analyse géographique

écosystème data complet et moderne :

- L'**architecture Medallion** garantit la qualité et la traçabilité des données tout au long du pipeline.
- **Apache Airflow** assure l'orchestration robuste et la supervision des tâches ETL.
- Le **scraping Selenium** et l'**API OAuth2** permettent la collecte multi-sources.
- **FastAPI** expose les données de manière performante et standardisée.
- Le **moteur NLP TF-IDF + Cosinus** offre une recommandation personnalisée efficace.
- **Power BI** apporte une dimension analytique et décisionnelle à l'écosystème.

Sur le plan **pédagogique**, ce projet a permis d'appréhender de manière concrète et intégrée les différentes facettes du métier de Data Engineer : ingestion de

données, transformation ETL, modélisation dimensionnelle, orchestration de pipelines, exposition via API, et visualisation BI. Il illustre également comment des technologies open-source (PostgreSQL, Apache Airflow, Selenium, FastAPI, React) peuvent être assemblées pour construire une plateforme data production-ready.

Annexe — Récapitulatif des Technologies Utilisées

Technologie	Catégorie	Usage dans le projet
Apache Air-flow	Orchestration	Planification et supervision du pipeline ETL
PostgreSQL	Base de données	Stockage des couches Bronze, Silver, Gold
Docker Compose	/ Infrastructure	Conteneurisation de tous les services
Selenium	Scraping	Extraction des offres LinkedIn
FastAPI	API	Exposition des données et du moteur NLP
Vite + React	Frontend	Interface utilisateur SPA
Pandas	ETL	Transformation et nettoyage des données
Scikit-learn	NLP/ML	TF-IDF vectorization et similarité cosinus
PyPDF2	Parsing	Extraction de texte depuis les CVs PDF
Power BI	BI	Dashboards et visualisations analytiques
Python	Langage	Langage principal du backend et ETL
JavaScript/JSX	Langage	Frontend React

Table 3 – *Stack technologique complète du projet Job Intelligent*